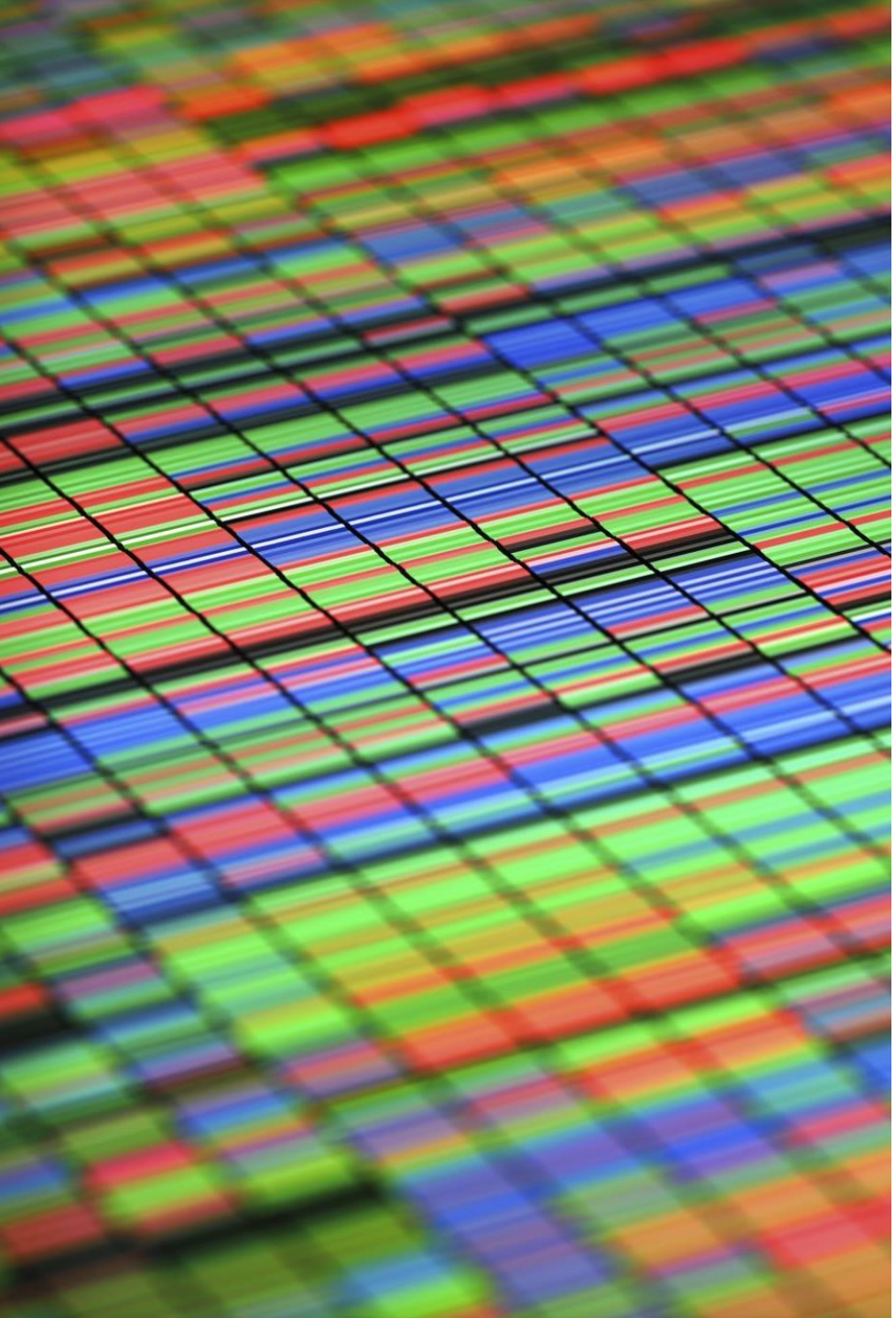


Estadística Descriptiva

ANÁLISIS DE DATOS NIVEL EXPLORADOR – MISIÓN 2

¿Qué es la media?

La media es el valor que representa, en promedio, a todos los datos de un conjunto, es como **repartir equitativamente** todos los datos para obtener un único valor que los represente.

¿Para qué sirve la media?

✓Para resumir un conjunto de datos con un solo número.

✓Para comparar grupos diferentes *(ej.: precio medio de vuelos entre diferentes ciudades).*

✓Para establecer patrones *(ej.: el ingreso medio en una región).*

Ejemplo cotidiano:

Imagina que haces una búsqueda en internet de tiquetes de avión de Medellín a Bogotá, y encuentras estos precios (en COP):

295.000 – 310.000 – 325.000 – 600.000 – 320.000

- Algunos son económicos, otros más caros. ¿Cuál sería un precio promedio?
- Si sumas todos y los repartes entre 5 vuelos, obtienes un precio promedio: la media.

¿Qué es la mediana?

La mediana es el **valor que queda justo en el medio** cuando los **datos están ordenados de menor a mayor**, a diferencia de la media, no se ve afectada por los valores extremos.

¿Para qué sirve la mediana?

- ✓Para entender el **valor central real** de un grupo.
- ✓Es más justa que la media cuando hay datos muy altos o bajos que distorsionan.
- ✓Es útil en situaciones donde hay valores extremos (outliers).

Ejemplo aplicado:

Supón que buscas precios de vuelos:

310.000 – 295.000 – 600.000 – 325.000 – 1.000.000

- La media será muy alta por los vuelos costosos.
- Pero si ordenas:
  - 295.000 – 310.000 – 325.000 – 600.000 – 1.000.000
  - El valor del medio es **325.000 → esa es la mediana**, y representa mejor el precio “típico”.

¿Qué es la moda?

La moda es el **valor que más se repite** en un conjunto de datos, es el dato **más común o más frecuente**.

¿Para qué sirve la moda?

- ✓Para saber qué valor se repite más (el más "popular").
- ✓Útil cuando se analiza:
  - ✓El número de estudiantes que eligen una carrera
  - ✓El producto más vendido
  - ✓La talla más común en una tienda
- ✓Funciona también con datos categóricos (texto), no solo con números.

Ejemplo simple:

Imagina que estos son los precios de 6 tiquetes de avión:

320.000 – 310.000 – 320.000 – 295.000 – 320.000 – 600.000

El valor 320.000 aparece 3 veces → esa es la moda.

¿Y si hay dos valores que se repiten igual?

320.000 – 310.000 – 320.000 – 310.000 – 295.000

Tanto 310.000 como 320.000 aparecen 2 veces → hay dos modas (distribución bimodal).

Desviación Estándar y Varianza

¿Qué miden estas dos?

Ambas miden cuánto se alejan los datos de la media.

- La varianza mide la dispersión total de los datos.
- La desviación estándar es la versión interpretable de esa dispersión (en las mismas unidades que los datos).

¿Por qué importa esto?

Si analizas edades de pacientes, notas de estudiantes o ingresos de personas, no solo te interesa el promedio.

✓Necesitas saber:
- ✓ ¿Están todos cerca del promedio?
- ✓ ¿O hay diferencias grandes entre unos y otros?

✓Por eso usamos la desviación estándar, que cuantifica cuán separados están los datos.

Ejemplo visual:

Supón que tienes dos listas de precios de tiquetes de avión (en COP):

➢**Grupo A** *(poco disperso):*320.000 – 325.000 – 330.000 – 325.000 – 320.000

➢**Grupo B** *(muy disperso):*100.000 – 250.000 – 500.000 – 600.000 – 900.000

Ambos grupos podrían tener un **promedio similar**, pero claramente:

✓En el grupo A, los precios están cerca de la media baja desviación.

✓En el grupo B, hay mucha variación entre precios alta desviación.

¿Qué es el rango?

El rango es la diferencia entre el valor más alto y el más bajo en un conjunto de datos.

Es una forma simple de saber qué tan extendidos están los datos.

¿Para qué sirve el rango?

- Para conocer la amplitud total de los datos.
- Es útil para:
  - Comparar **variabilidad** entre dos grupos
  - Detectar **diferencias extremas**
  - Identificar si puede haber **outliers** (valores atípicos)

Ejemplo sencillo:

Supón que tienes estos precios de vuelos (en COP):

310.000 – 325.000 – 299.000 – 600.000 – 295.000

- El más barato es **295.000**
- El más caro es **600.000**

El rango es:

600.000 – 295.000 = 305.000

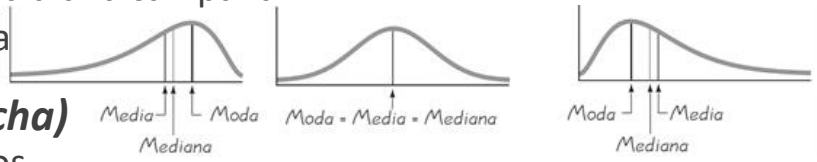
Sesgo y Curtosis

¿QUÉ ES EL SESGO (SKEWNESS)?

El sesgo indica si los datos están distribuidos simétricamente o si se inclinan hacia un lado (valores extremos a la izquierda o derecha).

Tipos de sesgo:

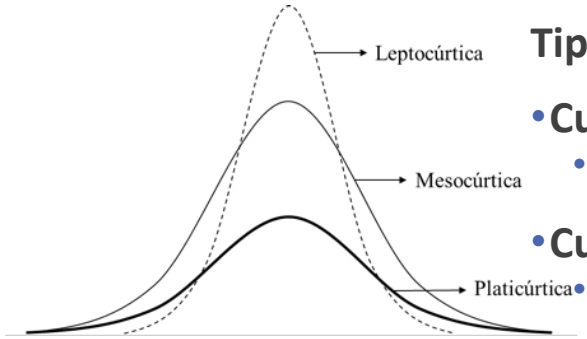
- ***Sesgo negativo (a la izquierda)***
  - Hay valores bajos extremos.
  - La media es menor que la mediana.
- ***Sesgo simétrico***
  - La distribución es parecida a una campana
  - Media ≈ Mediana ≈ Moda
- ***Sesgo positivo (a la derecha)***
  - Hay valores altos extremos.
  - La media es mayor que la mediana.



¿QUÉ ES LA CURTOSIS (KURTOSIS)?

La curtosis mide qué tan puntiaguda o plana es una distribución de datos.

Tipos de curtosis:



- **Curtosis normal (mesocúrtica)**
  - Tiene una forma equilibrada, como una campana regular.
- **Curtosis alta (leptocúrtica)**
  - Distribución con pico alto y extremos marcados (valores muy alejados de la media).
  - Más valores extremos (outliers).
- **Curtosis baja (platicúrtica)**
  - Distribución más plana, sin muchos valores extremos.
  - Los datos están más dispersos alrededor del centro.

Percentiles y Cuartiles

¿Qué son los percentiles?

Un percentil indica el valor por debajo del cual se encuentra un porcentaje específico de los datos.

¿Qué son los cuartiles?

Los cuartiles son valores que dividen los datos en cuatro partes iguales.

- Q1 (25%) = Primer cuartil = P25
- Q2 (50%) = Segundo cuartil = Mediana
- Q3 (75%) = Tercer cuartil = P75

¿Para qué sirven?

✓ Para **ver en qué parte de la distribución se encuentra un valor.**

✓ Para **detectar extremos** (valores muy altos o bajos).

✓ Son clave en el **boxplot**: un gráfico que muestra visualmente los cuartiles y los outliers.

Ejemplo aplicado

➢ En un colegio, si un estudiante está en el percentil 80:
  - ➢ Significa que superó al 80% de sus compañeros.

➢ En una empresa, si un salario está en el percentil 25:
  - ➢ Está entre el 25% más bajo de la compañía.

Actividad aplicada

Crea una tabla en Excel con 10 precios que consultaras en al web de vuelos entre las siguientes ciudades Medellin – Santa Marta. (ida: 30 nov // vuelta: 2 dic, 2 adultos), *Asegúrate de que los precios sean de distintos horarios o aerolíneas para que haya variación.*

Los estudiantes deberán hacer lo siguiente:

- Calculen media, mediana, moda, mínimo, máximo
- Saquen conclusiones con los compañeros que tienen al lado:
  - ¿Hay mucha diferencia entre el más barato y el más caro?
  - ¿Cuál precio representa mejor el “costo típico”?
  - ¿Hay valores extremos? ¿Cómo afectan la media?
  - ¿La mediana o la media representan mejor este caso?

Replicando análisis estadístico en Python

1. Importamos la librería de pandas
2. Creamos una lista llamada **“precios”** con los 10 precios de los vuelos consultados anteriormente.
3. Crear el DataFrame en base a la lista creada.

```
df = pd.DataFrame(precios, columns=["Precio"])
```

4. Hacer uso de las funciones de estadística con las que cuenta la librería pandas

```
print("Media:", df["Precio"].mean())
print("Mediana:", df["Precio"].median())
print("Moda:", df["Precio"].mode()[0])
print("Mínimo:", df["Precio"].min())
print("Máximo:", df["Precio"].max())
print("Rango:", df["Precio"].max() - df["Precio"].min())
print("Desviación estándar:", df["Precio"].std())
print("Sesgo (skew):", df["Precio"].skew())
print("Curtosis (kurt):", df["Precio"].kurt())
```

Limpieza y preparación de datos

¿Por qué limpiar los datos?

Cuando se trabaja con datos reales, no todo es tan ordenado como una tabla de Excel bien armada. Los datos muchas veces llegan **sucios, incompletos, desordenados o con errores**, lo cual puede afectar seriamente cualquier análisis que queramos hacer.

¿Qué pasa si no limpias los datos?

- Los promedios pueden salir distorsionados.
- Los gráficos pueden dar una imagen equivocada.
- Los modelos o decisiones se basarán en datos erróneos.
- Puedes llegar a conclusiones incorrectas.

¿Qué problemas puede haber en los datos?

1. **Faltan datos importantes**
   - Por ejemplo, una persona no respondió su edad en una encuesta.
2. **Datos incorrectos o imposibles**
   - Un paciente con 0 de presión arterial *(debería estar muerto).*
3. **Datos repetidos**
   - La misma persona registrada dos veces.
4. **Datos fuera de escala**
   - Un vuelo cuesta 3 millones y todos los demás 300 mil.
5. **Errores de formato**
   - Nombres escritos distinto *("Bogotá", "bogota", "BOGOTÁ").*



¿Por qué los datos nunca vienen perfectos?

Cuando imaginamos una base de datos, solemos pensar en **tablas limpias y ordenadas**, pero en la práctica, los datos vienen de **personas, sistemas, sensores, formularios, correos, etc.,** y todos esos orígenes tienen algo en común: **pueden fallar**.



¿Qué son los valores faltantes?

Un valor faltante es un dato que **debería estar, pero no está**. En pandas se representa como NaN (Not a Number).

¿Qué es NaN?

NaN significa "*Not a Number*" y se utiliza para representar valores faltantes o nulos en un conjunto de datos. Pandas y Numpy lo reconocen automáticamente y lo manejan de forma eficiente.

¿Cómo identificar valores faltantes?

Antes de limpiar los valores faltantes, primero hay que saber **dónde están, cuántos son, y en qué columnas aparecen.**

1. Ver visualmente en el DataFrame
   - Si cargas un archivo con pandas y ves celdas con NaN
2. Detectar valores faltantes con `.isnull()`
   - Devuelve un DataFrame del mismo tamaño pero con **`True`** donde hay valores faltantes y **`False`** donde no.
3. Visualmente usando `seaborn` + `matplotlib`
   - Grafica los espacios vacíos, esto ayuda a detectar de manera mas facil patrones.

Métodos para tratar valores faltantes

¿Qué significa *"tratar"* valores faltantes?

No podemos dejar los valores vacíos sin hacer nada. Tenemos que **decidir qué hacer con ellos:**

- *¿Los eliminamos?*
- *¿Los reemplazamos?*
- *¿Los dejamos como están?*

La decisión depende de cuántos hay, en qué columnas están y **cuánto afecta al análisis.**

Opción 1: **Eliminar los valores faltantes**	Cuando **hay muy pocos datos faltantes (menos del 5%).**
	Cuando la columna **no es muy importante.**
	Cuando **es mejor perder una fila que arriesgarse a suponer un valor incorrecto.**
Opción 2: **Imputar (rellenar) los valores faltantes**	Cuando **no puedes darte el lujo de eliminar datos (porque perderías demasiados).**
	Cuando tiene sentido **rellenar con un valor estimado o representativo.**



¿Qué son los datos atípicos?

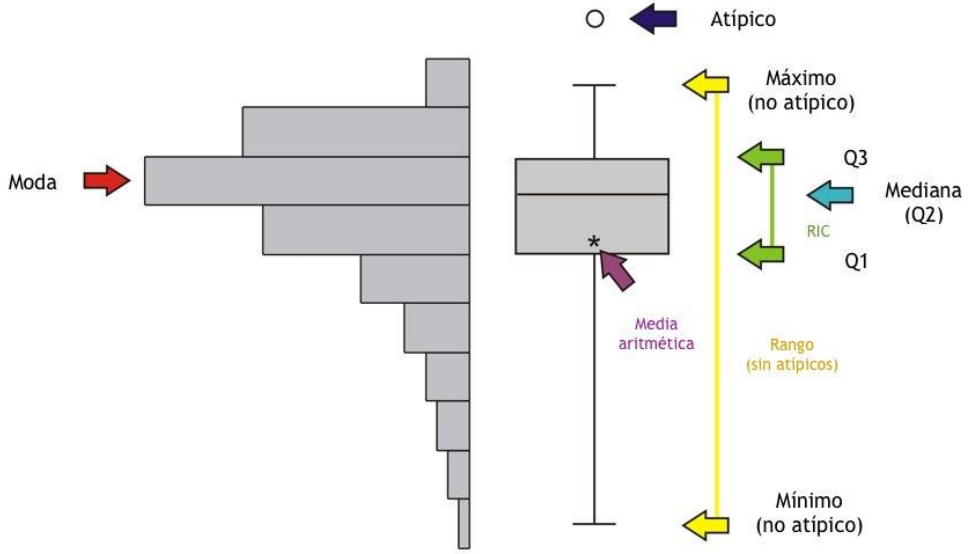
Un dato atípico ***(outlier)*** es un valor que **se aleja mucho del resto**. Es como un *"dato raro"* que **no sigue el mismo patrón que los demás**.

¿Por qué es importante detectarlos?

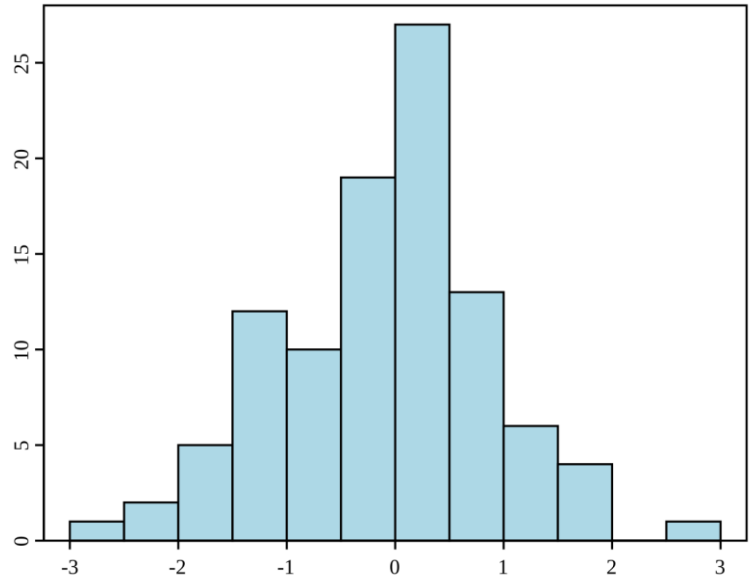
- **Pueden ser errores de ingreso de datos**
  - *Ejemplo:* alguien escribió 1200000 en lugar de 120000.
- **Pueden distorsionar resultados estadísticos**
  - La media sube mucho con un solo valor extremo.
- **A veces son casos interesantes**
  - Por ejemplo, el cliente que compra 100 veces más que los demás → no es error, ¡es VIP!

¿Cómo identificar valores atípicos?

- **Boxplot (diagrama de caja)**
  - Mediana (línea central)
  - Cuartiles (caja)
  - Límites normales (bigotes)
  - Outliers como puntos fuera de la caja



- **Histograma**
  - Sirve para ver si hay valores muy alejados del grupo principal.



¿Cómo manejar los datos atípicos?

Método 1: **Eliminación de outliers**

- Si el valor claramente **es un error.**
- Si hay **muy pocos outliers** y no afectan la muestra.
- Si los valores extremos **distorsionan los resultados y no aportan valor.**

Método 2: **Transformación**

- Cuando no se quiere eliminar el dato, pero sí **reducir su impacto.**
- Ideal para análisis estadístico o visualizaciones.
- Ejemplos:
  - ***Logaritmo:*** reduce la escala de valores muy grandes.
  - ***Escalamiento:*** convertir todos los valores a una misma escala.

Método 3: **Imputación**

- Cuando se sospecha que el outlier es **producto de un error**, pero no se puede dejar vacío.
- Se reemplaza por un valor más razonable (media, mediana, máximo aceptado).

DataFrame para práctica de valores faltantes

¿NaN, None y NA son lo mismo en Python?

"Aunque tengan nombres diferentes, None, NaN y pd.NA son **formas distintas de decir que falta un dato** y pandas sabe tratarlas como tales."

```
import pandas as pd
import numpy as np
# Diccionario de datos mixtos
datos = {
    "Nombre": ["Sofía", "Mateo", "Valentina", "Andrés", "Lucía", "Tomás"],
    "Edad": [25, 28, np.nan, 23, 30, 27],
    "Ciudad": ["Bogotá", "Cali", "Medellín", np.nan, "Cali", "Bogotá"],
    "Puntaje": [85, 90, 78, 92, np.nan, 88],
    "Asistencia": [20, 18, 19, np.nan, 17, 20],
    "Graduado": [True, False, True, False, True, np.nan]
}
# Crear DataFrame
df = pd.DataFrame(datos)
# Mostrar el DataFrame
print(df)
```

Detección y conteo de valores faltantes en un DataFrame

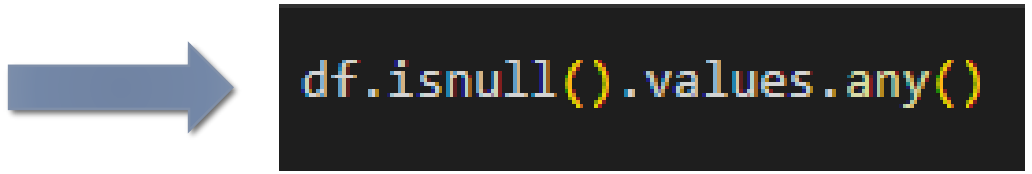
1. **Ver dónde hay valores faltantes con `.isnull()`**
   - Esto devuelve un nuevo DataFrame con el mismo tamaño que df, donde:
     - **`True`** indica que el valor está faltando (es NaN)
     - **`False`** indica que hay un valor presente



2. **Contar cuántos valores faltantes hay por columna con `.isnull().sum()`**
   - Esto devuelve una **serie de pandas** que indica cuántos valores faltantes hay en cada columna.



3. **Saber si hay al menos un valor faltante en todo el DataFrame**
   - Devuelve `True` si hay al menos un NaN **en todo el dataset**, útil para chequeo rápido.



4. **Saber cuántos valores faltantes hay en total**
   - Esto suma todos los valores faltantes en todo el DataFrame.



Eliminación de filas o columnas con valores faltantes

ELIMINAR FILAS QUE TIENEN AL MENOS UN VALOR FALTANTE

Esto elimina todas las filas que tengan al menos un **NaN**.

⚠ ***Importante:*** no modificar el DataFrame original

```
df_sin_nulos = df.dropna()
df_sin_nulos
```

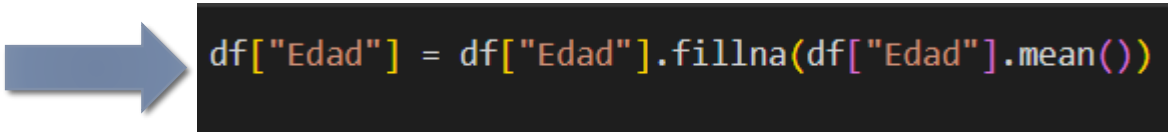
ELIMINAR COLUMNAS QUE TIENEN AL MENOS UN VALOR FALTANTE

Esto elimina todas las **columnas con algún valor faltante**.

```
df_sin_columnas_nulas = df.dropna(axis=1)
df_sin_columnas_nulas
```

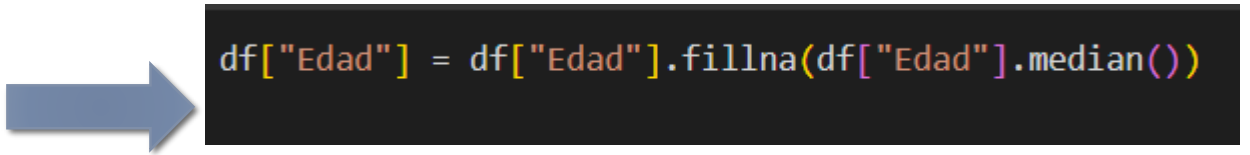
Imputación de valores faltantes

1. **Imputar con la media**
   - Útil para columnas numéricas cuando los valores están equilibrados.



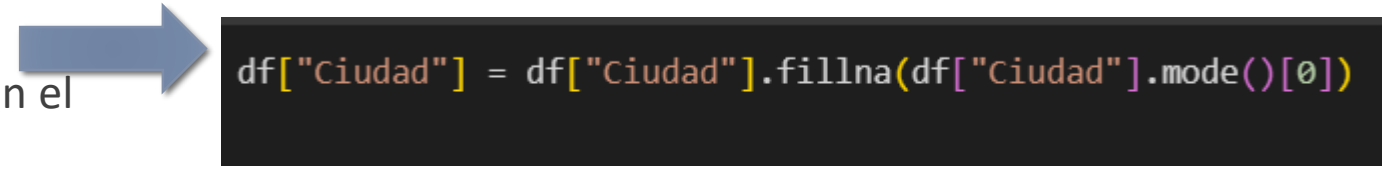
```
df["Edad"] = df["Edad"].fillna(df["Edad"].mean())
```

2. **Imputar con la mediana**
   - Recomendada cuando hay valores extremos que pueden alterar el promedio.



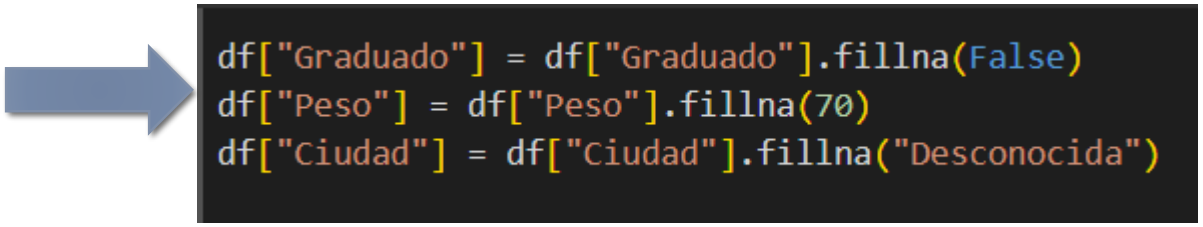
```
df["Edad"] = df["Edad"].fillna(df["Edad"].median())
```

3. **Imputar con la moda**
   - Ideal para columnas categóricas (texto, booleanos),donde queremos rellenar con el valor más común.



```
df["Ciudad"] = df["Ciudad"].fillna(df["Ciudad"].mode()[0])
```

4. **Imputar con un valor fijo personalizado**
   - Útil cuando el analista conoce el contexto y puede establecer un valor razonable.



```
df["Graduado"] = df["Graduado"].fillna(False)
df["Peso"] = df["Peso"].fillna(70)
df["Ciudad"] = df["Ciudad"].fillna("Desconocida")
```

Actividad: Limpieza de datos de consumo energético

```python
datos_energia = {
    "Ciudad": ["Bogotá", "Medellín", "Cali", "Barranquilla", "Bogotá", "Cali", "Medellín", np.nan],
    "Consumo_kWh": [350, 420, np.nan, 390, 310, 405, np.nan, 450],
    "Estrato": [3, 4, 3, np.nan, 2, 3, 4, 2],
    "Subsidio": [True, False, True, True, np.nan, True, False, True]
}
df = pd.DataFrame(datos_energia)
```

1. Detección y conteo

- Usa `.isnull()` y `.sum()` para contar nulos por columna.
- ¿Cuántos valores faltantes hay en total?

2. Eliminación

- Crea una copia del DataFrame y elimina:
  - Las filas que tengan al menos un NaN.
  - Las columnas que tengan al menos un NaN.
- ¿Cuánta información se pierde en cada caso?

3. Imputación

- Imputa los valores faltantes de estas formas:
  - **Consumo_kWh** → reemplazar con la media
  - **Estrato** → reemplazar con la mediana
  - **Ciudad** → reemplazar con la moda
  - **Subsidio** → reemplazar con un valor fijo (False)

Actividad: Limpieza y toma de decisiones

Dataset: `datos_energia_bootcamp.csv`

Carga el archivo `datos_energia_bootcamp.csv` en tu entorno de trabajo

1. Observa las primeras filas, revisa los nombres de las columnas, tipos de datos y estructura general.
2. Identifica los valores faltantes:
   - ¿Cuántos valores faltantes hay por columna?
   - ¿Alguna fila está completamente vacía?
3. La columna `"ID"` tiene más del 50% de valores faltantes.
   - ¿Qué representa esta columna? ¿Es útil conservarla?
4. ¿Qué columnas parecen más importantes para el análisis?
5. ¿Qué impacto podría tener dejar o eliminar los valores faltantes?
6. Decide qué columnas o filas eliminarías.
7. Decide en qué columnas tiene sentido imputar valores faltantes:
   - ¿Imputarías "Horas_Consumo" con la media? ¿Por qué sí o por qué no?
   - ¿Tiene sentido imputar "Ciudad" con la moda?
   - ¿Qué harías con los valores faltantes en "Subsidio"?

¿Por qué renombrar columnas?

Los nombres de las columnas son clave para la comprensión del análisis, renombrarlas permite que los datos sean más:

- ✓ *Claros*
- ✓ *Legibles*
- ✓ *Consistentes*
- ✓ *Profesionales*

¿Cuándo deberías renombrar?

- Antes de compartir datos con otros
- Antes de analizar o graficar
- Cuando importas archivos con nombres confusos o generados automáticamente
- Para facilitar el uso en código (evitar errores de sintaxis)

- Opción 1: **Renombrar solo algunas columnas**

```
df.rename(columns={
    "Consumo_kWh": "Consumo",
    "Horas_Consumo": "HorasUso"
}, inplace=True)
```

- Opción 2: **Renombrar todas las columnas de una vez**

```
df.columns = ["ID", "Ciudad", "Consumo", "Estrato", "Subsidio", "HorasUso"]
```

¿Por qué es importante cambiar los tipos de datos?

Cada columna en un DataFrame tiene un tipo de dato: *numérico, texto, booleano, fecha, etc*. Si ese tipo está mal definido, puede generar **errores en cálculos, filtros, o visualizaciones**.

Problemas por tipo de dato incorrecto:

- Un número guardado como texto: **`"45"`** en lugar de **45**
- Una fecha guardada como cadena de texto
- Un booleano como **`"Sí"`** y **`"No"`** en lugar de **`True`** y **`False`**

➢ Convertir **texto a número**

```
df["Edad"] = df["Edad"].astype(int)

df["Edad"] = df["Edad"].astype("Int64")
```

➢ Convertir a **fecha**

```
df["Fecha"] = pd.to_datetime(df["Fecha"])
```

➢ Convertir a **booleano**

```
df["Subsidio"] = df["Subsidio"].astype(bool)

df["Subsidio"] = df["Subsidio"].map({"Sí": True, "No": False})
```

¿Por qué eliminar columnas?

No todos los datos que llegan en un archivo son útiles para el análisis, algunas columnas pueden ser:

- *Redundantes*
- *Irrelevantes*
- *Altamente incompletas*
- *Usadas solo para identificación*

Eliminar columnas permite que el análisis sea más claro, **más rápido y más enfocado.**

- Eliminar una o más **columnas por nombre**

```
df.drop(columns=["Columna1", "Columna2"], inplace=True)
```

- Eliminar **columnas por posición**

```
df.drop(df.columns[[1, 3]], axis=1, inplace=True)
```

¿Por qué crear nuevas columnas?

Crear nuevas columnas permite **transformar los datos existentes, combinarlos, o crear variables** derivadas que son más útiles para el análisis.

Es una de las herramientas poderosas de un analista de datos porque:

✓ *Agrega valor sin necesidad de más datos externos.*

✓ *Permite crear indicadores, etiquetas o clasificaciones.*

✓ *Ayuda a facilitar el modelado o la visualización*.

1. A partir de operaciones entre columnas

```
df["Costo_Total"] = df["Consumo_kWh"] * 500
```

2. Crear etiquetas con **`np.where`**

```
import numpy as np
df["Tiene_Subsidio"] = np.where(df["Subsidio"] == True, "Sí", "No")
```

3. Extraer información de columnas existentes

```
df["Año"] = df["Fecha"].dt.year
```

Actividad: Análisis y preparación de datos de usuarios de telefonía

1. **Carga el archivo `JSON`**
   - Usa `pandas` para cargar el archivo `datos_telefonia_usuarios.json`
   - Visualiza las primeras filas.

2. **Explora y analiza**
   - Usa `.info()` y `.dtypes` para examinar los tipos de datos actuales.
   - Observa si hay inconsistencias, errores tipográficos, columnas mal nombradas, etc.

3. **Transforma y prepara los datos**
   - ¿Qué opinas del nombre actual de las columnas? ¿Son claras?
     - Renombra al menos 3 columnas para hacerlas más legibles o estándar.
   - ¿Hay columnas que no aportan al análisis?
     - Elimina cualquier columna que consideres irrelevante o poco útil para un análisis.
   - ¿Hay columnas que cambiarías por claridad o formato?
     - Cambia el tipo de dato de al menos 2 columnas si lo consideras necesario.
   - Crea al menos una nueva columna derivada de la información existente (*por ejemplo, operador + tipo*).

DataFrame para Identificación de outliers

- Crear un DataFrame simulado con consumo energético

```
import pandas as pd
import numpy as np

# Crear DataFrame con valores normales
datos = {
    "Cliente": [f"CL{i:03}" for i in range(1, 51)],
    "Consumo_kWh": np.append(np.random.normal(350, 40, 45).round(1), [200, 215, 700, 720, 750])
}

df = pd.DataFrame(datos)
```

- Calcular cuartiles e IQR

```
q1 = df["Consumo_kWh"].quantile(0.25)
q3 = df["Consumo_kWh"].quantile(0.75)
iqr = q3 - q1

# Límites para detectar outliers
limite_inferior = q1 - 1.5 * iqr
limite_superior = q3 + 1.5 * iqr

print("Q1:", q1)
print("Q3:", q3)
print("IQR:", iqr)
print("Límite inferior:", limite_inferior)
print("Límite superior:", limite_superior)
```

Detectar valores fuera del rango

```
outliers = df[(df["Consumo_kWh"] < limite_inferior) | (df["Consumo_kWh"] > limite_superior)]
print("Valores atípicos detectados:")
print(outliers)
```

Preguntas para reflexionar

- ¿Qué tan alejados están del resto?
- ¿Qué harías con ellos?

Tratamiento de valores atípicos

1. **Imputar los outliers**

```python
# Reemplazar los outliers por la mediana del consumo
mediana = df["Consumo_kWh"].median()
df_imputado = df.copy()
df_imputado.loc[
    (df["Consumo_kWh"] < limite_inferior) | (df["Consumo_kWh"] > limite_superior),
    "Consumo_kWh"
] = mediana
```

2. **Eliminar los outliers**

```python
# Filtrar solo los datos que están dentro del rango normal
df_sin_outliers = df[(df["Consumo_kWh"] >= limite_inferior) & (df["Consumo_kWh"] <= limite_superior)]
```

Actividad Integradora: Análisis y preparación de datos

- Dataset:
  `datos_criminalidad_actualizado.csv`

- Carga y exploración básica
  - Carga el archivo **`.csv`** con pandas
  - Observa las primeras y últimas filas
  - Usa **`.info()`**, **`.describe()`** para entender tipos de datos y estructura.

- Detección y tratamiento de valores faltantes
  - ¿Qué columnas contienen valores **NaN**?
  - ¿Qué estrategia aplicarías en cada caso? (eliminar, imputar, conservar)

- Identificación y tratamiento de valores atípicos
  - Usa el método del **rango intercuartílico (IQR**) sobre "tiempo_respuesta".
  - Detecta outliers y crea un nuevo DataFrame filtrado

- Preparación del dataset
  - Renombra columnas que tengan nombres poco claros o poco estándar
  - Cambia los tipos de datos si es necesario:
    - ¿Deberías convertir **"estrato"** o **"año"** a tipo entero?
    - ¿La columna **"fecha"** está en formato de fecha?
  - Elimina columnas si consideras que no aportan al análisis (por ejemplo, analiza **"id_caso"**).
  - Crea al menos una nueva columna, por ejemplo:
    - Clasificación del tipo de delito
    - Una columna binaria: ¿delito grave o no?